

tf.function Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/function

Compiles a function into a callable TensorFlow graph. (deprecated arguments) (deprecated arguments) (deprecated arguments)

Function Signatures

```
tf.function( func=None, input_signature=None, autograph=True,
jit_compile=None, reduce_retracing=False,
experimental_implements=None,
experimental_autograph_options=None,
experimental_attributes=None, experimental_relax_shapes=None,
experimental_compile=None,
experimental_follow_type_hints=None ) ->
tf.types.experimental.PolymorphicFunction
(experimental_compile)
(experimental_relax_shapes)
```

Code Examples

```
tf.function(  
    func=None,  
    input_signature=None,  
    autograph=True,  
    jit_compile=None,  
    reduce_retracing=False,  
    experimental_implements=None,  
    experimental_autograph_options=None,  
    experimental_attributes=None,  
    experimental_relax_shapes=None,  
    experimental_compile=None,  
    experimental_follow_type_hints=None  
) -> tf.types.experimental.PolymorphicFunction
```

```
tf.function(  
    func=None,  
    input_signature=None,  
    autograph=True,  
    jit_compile=None,  
    reduce_retracing=False,  
    experimental_implements=None,  
    experimental_autograph_options=None,  
    experimental_attributes=None,  
    experimental_relax_shapes=None,  
    experimental_compile=None,  
    experimental_follow_type_hints=None  
) -> tf.types.experimental.PolymorphicFunction
```

```
tf.types.experimental.PolymorphicFunction
```

```
@tf.function  
def f(x, y):  
    return x ** 2 + y  
x = tf.constant([2, 3])  
y = tf.constant([3, -2])  
f(x, y)
```

```
@tf.function
```

```
def f(x, y):
```

```
    return x ** 2 + y
```

```
x = tf.constant([2, 3])
```

Documentation

Compiles a function into a callable TensorFlow graph. (deprecated arguments)
(deprecated arguments) (deprecated arguments)

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.function`

Used in the notebooks

- Better performance with `tf.function`
 - Introduction to graphs and `tf.function`
 - Extension types
 - Random number generation

- Import a JAX model using JAX2TF
- Custom training with `tf.distribute.Strategy`
- Distributed Input
- Distributed training with DTensors
- Parameter server training with `ParameterServerStrategy`
- Using DTensors with Keras

`tf.function` constructs a `tf.types.experimental.PolymorphicFunction` that executes a TensorFlow graph (`tf.Graph`) created by trace-compiling the TensorFlow operations in `func`. More information on the topic can be found in [Introduction to Graphs and `tf.function`](#).

See [Better Performance with `tf.function`](#) for tips on performance and known limitations.

Example usage:

The trace-compilation allows non-TensorFlow operations to execute, but under special conditions. In general, only TensorFlow operations are guaranteed to run and create fresh results whenever the `PolymorphicFunction` is called.

Features

`func` may use data-dependent Python control flow statements, including `if`, `for`, `while`, `break`, `continue` and `return`:

`func`'s closure may include `tf.Tensor` and `tf.Variable` objects:

`func` may also use ops with side effects, such as `tf.print`, `tf.Variable`

and others:

Instead, use TensorFlow collections like `tf.TensorArray`:

`tf.function` creates polymorphic callables

Internally, `tf.types.experimental.PolymorphicFunction` may contain multiple `tf.types.experimental.ConcreteFunctions`, each specialized to arguments with different data types or shapes, since TensorFlow can perform more optimizations on graphs of specific shapes, dtypes and values of constant arguments. `tf.function` treats any pure Python values as opaque objects (best thought of as compile-time constants), and builds a separate `tf.Graph` for each set of Python arguments that it encounters.

For more information, see the [tf.function guide](#)

Executing a `PolymorphicFunction` will select and execute the appropriate `ConcreteFunction` based on the argument types and values.

To obtain an individual `ConcreteFunction`, use the `PolymorphicFunction.get_concrete_function` method. It can be called with the same arguments as `func` and returns a `tf.types.experimental.ConcreteFunction`. `ConcreteFunctions` are backed by a single `tf.Graph`:

`ConcreteFunctions` can be executed just like `PolymorphicFunctions`, but their input is restricted to the types to which they're specialized.

Retracing

`ConcreteFunctions` are built (traced) on the fly, as the `PolymorphicFunction` is called with new TensorFlow types or shapes, or with new Python values as

arguments. When `PolymorphicFunction` builds a new trace, it is said that `func` is retraced. Retracing is a frequent performance concern for `tf.function` as it can be considerably slower than executing a graph that's already been traced. It is ideal to minimize the amount of retracing in your code.

Python numerical arguments should only be used when they take few distinct values, such as hyperparameters like the number of layers in a neural network.

Input signatures

For Tensor arguments, `PolymorphicFunction` creates a new `ConcreteFunction` for every unique set of input shapes and datatypes. The example below creates two separate `ConcreteFunctions`, each specialized to a different shape:

An "input signature" can be optionally provided to `tf.function` to control this process. The input signature specifies the shape and type of each Tensor argument to the function using a `tf.TensorSpec` object. More general shapes can be used. This ensures only one `ConcreteFunction` is created, and restricts the `PolymorphicFunction` to the specified shapes and types. It is an effective way to limit retracing when Tensors have dynamic shapes.

Variables may only be created once

`tf.function` only allows creating new `tf.Variable` objects when it is called for the first time:

In general, it is recommended to create `tf.Variables` outside of `tf.function`.

In simple cases, persisting state across `tf.function` boundaries may be implemented using a pure functional style in which state is represented by `tf.Tensors` passed as arguments and returned as return values.

Contrast the two styles below:

Python operations execute only once per trace

func may contain TensorFlow operations mixed with pure Python operations. However, when the function is executed, only the TensorFlow operations will run. The Python operations run only once, at trace time. If TensorFlow operations depend on results from Python operations, those results will be frozen into the graph.

Returns

Raises